Institute of Geographical Information Systems (IGIS)

School of Civil & Environmental Engineering (SCEE)

National University of Science and Technology (NUST)

LAB # 7: Open Ended Lab

Class:	BE-2024	**Semester:**	Spring 2026
Course:	CS-250 Data Structures and Algorithms	**Date:**	11th March 2026
Regn. No:	**526123**	**Max. Time:**	**1 week**

Implementing Queue ADT Using Arrays and Linked Lists

Implementing Queue ADT Using Arrays and Linked Lists 0
Restaurant Order Queue Using Array Implementation Output 1
Printer Queue Using Linked List Implementation Output 2
Explanation of Implementation 3
Output Explanation 3
Challenges Encountered 3
Practical Scenario Reflection 3
Restaurant Order System 3
Printer Job Queue 4
Advanced Considerations 4
Priority Orders for VIP Customers 4
Real-Life Improvement 4
Personal Reflection 4
Most Challenging Part 4
Learning Outcome 4

Restaurant Order Queue Using Array Implementation Output

settings.json | task1 - Array Implementation.cpp

DSA > LAB-07:OEL > task1 - Array Implementation.cpp > main()

```cpp
#include <iostream>
#include <string>
using namespace std;

#define SIZE 10

struct Order
{
    int orderID;
    string customerName;
    string food;
    bool drink;
    string status;
};

class RestaurantQueue
{
private:
    Order queue[SIZE];
    int front;
    int rear;

public:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
alinawaz@Alis-MacBook-Air LAB-07:OEL % g++ task1\ -\ Array\ Implementation.cpp -o task1
alinawaz@Alis-MacBook-Air LAB-07:OEL % ./task1
Order added for Ali
Order added for M Hamza
Order added for M Muaaz

Current Orders:
Order ID: 1
Customer: Ali
Food: Burger
Drink: Yes
Status: Pending
Order ID: 2
Customer: M Hamza
Food: Pizza
Drink: No
Status: Pending
Order ID: 3
Customer: M Muaaz
Food: Pasta
Drink: Yes
Status: Pending

Processing orders...
Processing Order ID: 1
Processing Order ID: 2

Current Orders:
Order ID: 3
Customer: M Muaaz
Food: Pasta
Drink: Yes
Status: Pending
alinawaz@Alis-MacBook-Air LAB-07:OEL %
```

0 0

Printer Queue Using Linked List Implementation Output

settings.json | task1 - Array Implementation.cpp | *task2 - LinkedList Implementation.cpp*

DSA > LAB-07:OEL > task2 - LinkedList Implementation.cpp > main()

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int jobID;
    Node *next;
};

class PrinterQueue
{
private:
    Node *front;
    Node *rear;

public:
    PrinterQueue()
    {
        front = rear = NULL;
    }

    bool isEmpty()
    {
        return front == NULL;
    }

    void enqueue(int id)
    {
        Node *newNode = new Node;
        newNode->jobID = id;
        newNode->next = NULL;

        if (isEmpty())
        {
            front = rear = newNode;
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
alinawaz@Alis-MacBook-Air LAB-07:OEL % g++ task2\ -\ LinkedList\ Implementation.cpp -o task2
alinawaz@Alis-MacBook-Air LAB-07:OEL % ./task2
Print Job 101 added
Print Job 102 added
Print Job 103 added

Current Print Jobs:
Job ID: 101
Job ID: 102
Job ID: 103

Processing jobs...
Printing Job ID: 101
Printing Job ID: 102

Current Print Jobs:
Job ID: 103
alinawaz@Alis-MacBook-Air LAB-07:OEL %
```

Explanation of Implementation

In this lab, the Queue Abstract Data Type (ADT) was implemented using two methods: arrays and linked lists.

In the restaurant order system, an array-based queue was used to manage customer orders. Each order contains an order ID, customer name, food item, drink option, and order status. Orders are added using the enqueue operation and processed using the dequeue operation following the First-In-First-Out principle.

In the printer queue simulation, a linked list implementation was used. Each print job is stored as a node containing the job ID and a pointer to the next job. This allows the queue to grow dynamically without fixed size limitations.

Output Explanation

The program first adds multiple orders to the restaurant queue.

Then the system processes some orders and displays the remaining orders.

Similarly, the printer queue simulation adds print jobs and processes them sequentially.

Screenshots of program output demonstrate that jobs are processed in the same order they arrive.

Challenges Encountered

One challenge was correctly managing the front and rear pointers in the queue. If these pointers are not updated properly, the queue can behave incorrectly.

Another challenge was ensuring memory is properly managed in the linked list implementation. This was solved by deleting nodes after processing print jobs.

Practical Scenario Reflection

Restaurant Order System

Using a queue improves efficiency because orders are handled in the exact order they arrive. This ensures fairness among customers and prevents confusion in the kitchen. It also helps the restaurant staff track pending orders easily.

Printer Job Queue

A linked list allows the printer queue to grow dynamically. Unlike an array queue with fixed size, new print jobs can always be added without worrying about memory limits.

Advanced Considerations

Priority Orders for VIP Customers

The queue could be modified into a **priority queue**. VIP orders would be inserted at the front of the queue so they are processed before normal orders.

Real-Life Improvement

In the restaurant system, a digital order tracking system could be added to show the current order status (Pending, Cooking, Served, Completed) on a display screen for customers.

Personal Reflection

Most Challenging Part

The most challenging part was managing the queue pointers correctly and ensuring that enqueue and dequeue operations maintain the correct order.

Learning Outcome

This lab improved understanding of how data structures like queues are applied in real-world systems such as restaurant management and printer scheduling. It demonstrated how FIFO structures help manage tasks efficiently.